EinfŸhrung in die C#-Programmierung

Version 1.0.0, 30. Januar 2026

Author: CHA, SLM | C#-Adaption: Kimi K2.5 | Prompter: Georg Graf (GRG)

Inhaltsverzeichnis

1. Grundlagen . . . 1
   1.1. Entwicklungsumgebung JetBrains Rider . . . 1
   1.2. Alternative Entwicklungsumgebungen . . . 2
2. Namens- und Coding-Konventionen . . . 3
   2.1. Allgemeine GrundsŠtze . . . 3
   2.2. Namenskonventionen . . . 3
   2.3. Code-Formatierung . . . 3
   2.4. Fail-Fast Pattern . . . 4
   2.5. Kommentare und Dokumentation . . . 5
   2.6. Verzeichnisstruktur und Organisation . . . 5
3. Professionelle Entwicklungswerkzeuge . . . 6
   3.1. Professionelle IDEs: Rider und Visual Studio . . . 6
   3.2. Projektstruktur und Solutions . . . 6
   3.3. Debugger . . . 6
4. Unit-Tests und TDD (xUnit) . . . 7
   4.1. Test-Driven Development (TDD) . . . 7
   4.2. Testprojekt einrichten . . . 7
   4.3. Facts und Theories . . . 7
   4.4. Setup und Lifecycle . . . 8
   4.5. Wichtige Assertions . . . 8
   4.6. Tests ausfŸhren . . . 8
5. Klassen, Objekte und Methoden . . . 10
   5.1. Properties (Eigenschaften) . . . 11
   5.2. Auto-Properties . . . 12
   5.3. Kommentare . . . 12
   5.4. Konstruktoren . . . 12
   5.5. Methoden . . . 13
   5.6. Signatur von Methoden . . . 14
   5.7. Parameter-PrŸfungen (Fail-Fast) . . . 15
   5.8. Lokale Variablen, Berechnungen und Typkonvertierung . . . 15
6. Variablen, Datentypen und Methoden . . . 17
   6.1. Werttypen . . . 17
   6.2. Referenztypen . . . 17
   6.3. Methoden (Fortsetzung) . . . 18
   6.4. ParameterŸbergabe mit `ref` und `out` . . . 19
   6.5. Wann ist die Verwendung von `ref` und `out` sinnvoll? . . . 19
7. Objekt-Beziehungen . . . 20
8. Strings . . . 22
   8.1. String-Vergleich . . . 23
   8.2. StringBuilder . . . 23
9. Schleifen . . . 24
   9.1. `while`-Schleife . . . 24
   9.2. `do-while`-Schleife . . . 24
   9.3. `for`-Schleife . . . 24
   9.4. `foreach`-Schleife . . . 24

9.5. Codebeispiele und Code-Wiederverwendung . . . 24
10. Arrays und Collections . . . 28
10.1. Arrays . . . 28
10.2. Generische Listen (List<T>) . . . 28
10.3. Weitere Collection-Typen . . . 28
10.4. Manuelle Array-Verwaltung . . . 29
11. Statische Methoden und NŸtzliche Klassen . . . 31
11.1. Statische Methoden . . . 31
11.2. Die `Main`-Methode . . . 31
11.3. NŸtzliche Klassen: Math und Random . . . 31
12. Vererbung . . . 33
12.1. Abstrakte Klassen und Methoden . . . 34
13. Interfaces . . . 35
14. Datenbanken mit Entity Framework Core (EF Core) . . . 36
14.1. EF Core Pakete installieren . . . 36
14.2. Das DomŠnenmodell und der DbContext . . . 36
14.3. Schema-Migrations . . . 36
14.4. CRUD-Operationen . . . 37
14.5. 1:n Beziehungen (One-to-Many) . . . 37
14.6. n:m Beziehungen (Many-to-Many) . . . 38
14.7. Laden von Beziehungen (Eager Loading) . . . 38
14.8. Best Practices fŸr EF Core . . . 39
15. LINQ (Language Integrated Query) . . . 40
15.1. Grundlegende LINQ-Operationen . . . 40
15.2. LINQ with Objects . . . 40
15.3. Query-Syntax vs. Method-Syntax . . . 41
16. Exception Handling . . . 42
16.1. Grundlagen . . . 42
16.2. Eigene Exceptions . . . 42
17. Razor . . . 43
17.1. Razor-Grundlagen . . . 43
17.2. Razor-Page Model . . . 43
17.3. Razor-Komponenten . . . 44
18. Razor mit HTMX . . . 45
18.1. Was ist HTMX? . . . 45
18.2. HTMX in Razor Pages . . . 45
18.3. PageModel fŸr HTMX . . . 46
18.4. HTMX-Partial Views . . . 47
18.5. HTMX-Trigger und Events . . . 47
18.6. HTMX mit Animationen . . . 48
18.7. Vorteile von Razor + HTMX . . . 48
18.8. Best Practices fŸr Razor + HTMX . . . 49

Kapitel 1. Grundlagen

Dieses Skriptum beinhaltet die Grundlagen der Programmierung in der Programmiersprache C#. Programmieren durchdringt unser Leben heute in vielen Bereichen, die resultierende Software findet sich in zahlreichen Bereichen: mobile Applikationen auf Smartphones und Tablets, Webanwendungen, Spiele, Server-Anwendungen, Cloud-Computing u.v.m. Die hier vorgestellten Konzepte stellen die Grundlage fŸr all jene Anwendungsgebiete dar, der primŠre Fokus liegt auf der Entwicklung von *Backends*, also Programmen die auf Servern laufen und die *Frontends* (also die Clients) mit den entsprechenden Daten und Berechnungen versorgen.

Programmierung bedeutet grundsŠtzlich, eine Folge von Operationen auf einem Mikroprozessor (CPU) anzusteuern, welche die Daten im Hauptspeicher oder Permanentspeicher verarbeiten. Die Programmierung auf Prozessorebene (Assembler-Sprachen) ist jedoch sehr aufwŠndig, und wird heutzutage nur noch fŸr wenige Spezialanwendungen genutzt. Die sogenannten hšheren Programmiersprachen wie beispielsweise Java, C# oder Python setzten auf einer hšheren Abstraktionsebene an, die mehr dem menschlichen Denken entspricht, und eine schnellere und robustere Programmentwicklung ermšglicht.

Das Programm besteht dabei zunŠchst aus dem Quelltext oder Code, d.h. ein oder mehrere Dateien im reinen Text-Format. Um das Programm tatsŠchlich ausfŸhren zu kšnnen, muss es zunŠchst *Ÿbersetzt* werden. Diese Aufgabe Ÿbernimmt der *Compiler*. EnthŠlt der Programmcode Fehler, konkret Elemente die nicht der Vorgabe der Programmiersprache entsprechen, so werden entsprechende Fehlermeldungen erzeugt, die dem Programmierer oder der Programmiererin bei der Behebung ebendieser helfen. Konnte das Programm zur GŠnze Ÿbersetzt (compiliert) werden, so kann es anschlie§end ausgefŸhrt werden. Die reine compilier-fŠhigkeit des Programmes sagt natŸrlich noch nichts Ÿber seine Korrektheit aus. Viele Fehler machen sich erst zur Laufzeit bemerkbar, und mŸssen anschlie§end behoben werden. In der modernen Programmierung versucht man die Anzahl der Fehler in Programmen durch *Tests* zu minimieren. Das AufspŸren von Fehlern oder unerwartetem Verhalten in Programmen wird durch *Debugger* unterstŸtzt, welche es ermšglichen das Programm an bestimmten Stellen anzuhalten und aktuelle Werte zu analysieren.

Bei C# wird der Programmcode vom Compiler in eine Zwischensprache (IL - Intermediate Language) Ÿbersetzt. Diese wird dann auf der Common Language Runtime (CLR), einer virtuellen Maschine, ausgefŸhrt. Dieses Konzept ermšglicht es die resultierenden Programme auf verschiedenen Betriebssystemen und Hardware-Architekturen auszufŸhren (dank .NET Core/.NET 5+).

Die Werkzeuge Compiler, Debugger, Text-Editor und vieles mehr werden Ÿblicherweise in einer sogenannten *Entwicklungsumgebung* (IDE - Integrated Development Environment) verwendet. Diese ermšglicht einen komfortablen und intuitiven Umgang mit allen benštigten Werkzeugen. Wir verwenden im Unterricht primŠr JetBrains Rider.

1.1. Entwicklungsumgebung JetBrains Rider

JetBrains Rider ist eine plattformunabhŠngige IDE, die auf der IntelliJ-Plattform und ReSharper basiert. Sie bietet eine exzellente UnterstŸtzung fŸr .NET-Entwicklung auf Windows, macOS und Linux.

Key Points: * Intelligente Code-Analyse: Erkennt Fehler bereits wŠhrend des Tippens und schlŠgt Optimierungen vor (basierend auf der ReSharper-Engine). * Refactoring: MŠchtige Werkzeuge zum sicheren Umbenennen oder Verschieben von Code-Teilen. * Integrierte Werkzeuge: Debugger, Unit-Test-Runner und Git-UnterstŸtzung sind nahtlos integriert. * Performance: Schnelle Indizierung und flŸssiges Arbeiten auch in gro§en Projekten.

1.2. Alternative Entwicklungsumgebungen

Je nach Anwendungsfall oder persšnlicher PrŠferenz kšnnen auch andere IDEs verwendet werden:

1.2.1. Microsoft Visual Studio

Der Industriestandard fŸr die Windows-Entwicklung. * Key Points: Umfassendste UnterstŸtzung fŸr Windows-spezifische Technologien (WPF, WinForms), sehr mŠchtiger Debugger, jedoch nur fŸr Windows verfŸgbar.

1.2.2. Visual Studio Code (VS Code)

Ein leichtgewichtiger, plattformunabhŠngiger Editor, der durch Plugins zur vollwertigen Entwicklungsumgebung ausgebaut werden kann. * Key Points: Extrem schnell, sehr flexibel und kostenlos. * Empfohlene Plugins fŸr C#: C# Dev Kit: Die offizielle Erweiterung von Microsoft fŸr Solution-Management und Testing. C#: Bietet Basis-SprachunterstŸtzung, IntelliSense und Debugging. .NET Install Tool:** Hilft bei der Verwaltung der benštigten .NET SDKs.

1.2.3. Projekt erstellen mittels .NET CLI

UnabhŠngig von der gewŠhlten IDE kšnnen Projekte Ÿber die Kommandozeile (Terminal) erstellt werden:

```
# Erstellen eines neuen Konsolenprojekts
dotnet new console -n MeinProjekt
cd MeinProjekt

# Projekt šffnen
code .
```

In VS Code šffnet sich das Projekt. Die Datei `Program.cs` enthŠlt den Einstiegspunkt des Programms. Sie kšnnen das Programm mit `F5` oder Ÿber das MenŸ "Run" starten.

Kapitel 2. Namens- und Coding-Konventionen

Bevor wir mit der eigentlichen Programmierung beginnen, ist es wichtig, sich mit den gŠngigen Konventionen vertraut zu machen. Diese Konventionen werden zwar vom Compiler nicht erzwungen, sind aber unbedingt einzuhalten, um lesbaren, wartbaren und professionellen Code zu schreiben.

2.1. Allgemeine GrundsŠtze

- Konsistenz: Verwenden Sie einen Stil konsequent durchgehend im gesamten Projekt
- Klarheit: Code sollte selbsterklŠrend sein - gute Namen sind wichtiger als Kommentare
- Einfachheit: Bevorzugen Sie einfache, direkte Lšsungen gegenŸber cleveren, komplizierten
- Fail-Fast: Validieren Sie Eingaben so frŸh wie mšglich und werfen Sie Exceptions bei ungŸltigen Werten

2.2. Namenskonventionen

Element	Konvention	Beispiel
Klassen	PascalCase	`Student`, `MitarbeiterRepository`
Interfaces	PascalCase, beginnt mit I	`IRepository`, `IStudentService`
Methoden	PascalCase	`GetName()`, `BerechneAlter()`
Eigenschaften (Properties)	PascalCase	`Name`, `Geburtsjahr`
Felder (private)	camelCase mit Unterstrich	`_name`, `_geburtsjahr`
Konstanten	PascalCase oder UPPER_SNAKE_CASE	`MaxGroesse` oder `MAX_GROESSE`
Parameter	camelCase	`neuerName`, `geburtsjahr`
Lokale Variablen	camelCase	`ergebnis`, `zaehler`
Enum-Typen	PascalCase	`Status`, `Farbe`
Enum-Werte	PascalCase	`Aktiv`, `Inaktiv`

2.3. Code-Formatierung

```
// Beispiel fŸr korrekte Formatierung
public class Student
{
Ê   private string _name;
Ê   private int _geburtsjahr;

Ê   public Student(string name, int geburtsjahr)
Ê   {
Ê       if (string.IsNullOrWhiteSpace(name))
Ê           throw new ArgumentException("Name darf nicht leer sein", nameof(name));
Ê       if (geburtsjahr < 1900 || geburtsjahr > DateTime.Now.Year)
Ê           throw new ArgumentOutOfRangeException(nameof(geburtsjahr));

Ê       _name = name;
Ê       _geburtsjahr = geburtsjahr;
Ê   }

Ê   public string Name
Ê   {
```

```
Ê       get => _name;
Ê       set
Ê       {
Ê           if (string.IsNullOrWhiteSpace(value))
Ê               throw new ArgumentException("Name darf nicht leer sein", nameof(value));
Ê           _name = value;
Ê       }
Ê   }

Ê   public int Geburtsjahr => _geburtsjahr;

Ê   public int BerechneAlter(int aktuellesJahr)
Ê   {
Ê       if (aktuellesJahr < _geburtsjahr)
Ê           throw new ArgumentException("Aktuelles Jahr muss nach Geburtsjahr liegen");

Ê       return aktuellesJahr - _geburtsjahr;
Ê   }
}
```

Wichtige Formatierungsregeln:

- EinrŸckung: 4 Leerzeichen pro Ebene (keine Tabs)
- Klammern: ...ffnende Klammer in neuer Zeile (Allman-Style)
- Leerzeichen: Nach SchlŸsselwšrtern (`if`, `for`, `while`), vor und nach Operatoren
- ZeilenlŠnge: Maximal 120 Zeichen pro Zeile
- Leerzeilen: Trennen Sie logische Code-Blšcke mit Leerzeilen

2.4. Fail-Fast Pattern

Das Fail-Fast-Pattern besagt, dass Fehler so frŸh wie mšglich erkannt und behandelt werden sollen. Das bedeutet:

1. Parameter-Validierung: PrŸfen Sie Parameter sofort am Beginn einer Methode
2. Exceptions werfen: Bei ungŸltigen Werten sofort eine Exception werfen, nicht nur eine Fehlermeldung ausgeben
3. Keine Stille Fehler: Vermeiden Sie, dass ungŸltige Werte stillschweigend korrigiert werden
4. Guard Clauses: Verwenden Sie frŸhe RŸckgaben oder Exceptions um verschachtelte If-Blšcke zu vermeiden

Vergleich Happy-Path vs. Fail-Fast:

```
// Happy-Path (NICHT empfohlen) - als Property
public int Groesse
{
Ê   get => _groesse;
Ê   set
Ê   {
Ê       if (value >= 110 && value <= 220)
Ê       {
Ê           _groesse = value;
Ê       }
Ê       else
Ê       {
Ê           Console.WriteLine("Fehler: Grš§e ungŸltig");
```

```
Ê            _groesse = 170; // Stille Korrektur
Ê        }
Ê    }
}

// Fail-Fast (empfohlen) - als Property mit Guard Clause
public int Groesse
{
Ê    get => _groesse;
Ê    set
Ê    {
Ê        if (value < 110 || value > 220)
Ê            throw new ArgumentOutOfRangeException(nameof(value),
Ê                "Grš§e muss zwischen 110 und 220 cm liegen");

Ê        _groesse = value;
Ê    }
}
```

2.5. Kommentare und Dokumentation

Verwenden Sie XML-Dokumentation fŸr šffentliche APIs:

```
/// <summary>
/// Berechnet das Alter einer Person basierend auf dem aktuellen Jahr.
/// </summary>
/// <param name="aktuellesJahr">Das aktuelle Jahr</param>
/// <returns>Das Alter in Jahren</returns>
/// <exception cref="ArgumentException">Wenn aktuellesJahr vor dem Geburtsjahr liegt</exception>
public int BerechneAlter(int aktuellesJahr)
{
Ê    if (aktuellesJahr < _geburtsjahr)
Ê        throw new ArgumentException("Aktuelles Jahr muss nach Geburtsjahr liegen");

Ê    return aktuellesJahr - _geburtsjahr;
}
```

2.6. Verzeichnisstruktur und Organisation

Organisieren Sie Ihr Projekt nach dem folgenden Muster:

```
MeinProjekt/
!"" src/
#   $"" MeinProjekt/
#       !"" Models/           # DomŠnenmodelle
#       !"" Services/         # GeschŠftslogik
#       !"" Repositories/     # Datenzugriff
#       !"" Exceptions/       # Eigene Exception-Typen
#       !"" Utils/            # Hilfsfunktionen
#       $"" Program.cs        # Einstiegspunkt
!"" tests/
#   $"" MeinProjekt.Tests/
$"" MeinProjekt.sln
```

Kapitel 3. Professionelle Entwicklungswerkzeuge

3.1. Professionelle IDEs: Rider und Visual Studio

FŸr die professionelle C#-Entwicklung sind JetBrains Rider und Microsoft Visual Studio die fŸhrenden Werkzeuge. Beide bieten einen enormen Funktionsumfang, der Ÿber einfache Texteditoren weit hinausgeht.

Gemeinsame Features: * IntelliSense / Code Completion: Intelligente Code-VervollstŠndigung mit Kontextinformationen. * Refactoring: Automatisches Umbenennen, Extrahieren von Methoden oder Klassen, sowie sicheres Verschieben von Code-Teilen. * Fortgeschrittener Debugger: Haltepunkte mit Bedingungen, †berwachung von Variablen (Watch) und Analyse des Call-Stacks. * NuGet-Integration: Einfache Verwaltung von externen Bibliotheken Ÿber den Paketmanager. * Test Runner: Komfortable OberflŠchen zum AusfŸhren und Verwalten von Unit-Tests (xUnit, NUnit).

3.2. Projektstruktur und Solutions

In der .NET-Welt werden Projekte Ÿblicherweise in Solutions (`.sln`) organisiert. Eine Solution kann mehrere Projekte enthalten (z.B. das Hauptprogramm und ein separates Testprojekt).

Ein typisches .NET-Projekt hat folgende physische Struktur:

```
MeineLoesung/
!"" MeineLoesung.sln           # Solution-Datei
!"" MeinProjekt/               # Projektverzeichnis
#   !"" MeinProjekt.csproj     # Projektdatei
#   !"" Program.cs             # Einstiegspunkt
#   $"" ...                    # Weitere Quellcodedateien
$"" MeinProjekt.Tests/         # Testprojekt
Ê   $"" ...
```

3.3. Debugger

Der Debugger ermšglicht es, die AusfŸhrung des Programmes anzuhalten, um den aktuellen Zustand (also die aktuellen Werte der Variablen und Eigenschaften) zu ŸberprŸfen. Diese Methode ermšglicht Fehler relativ einfach zu lokalisieren und zu beheben.

In modernen IDEs wie JetBrains Rider, Visual Studio oder VS Code setzt man dazu am linken Rand neben der Zeilennummer im Code-Editor einen sogenannten *Breakpoint* (Haltepunkt). Die nŠchste AusfŸhrung des Programmes stoppt dann bei genau dieser Codezeile. Das Debugger-Fenster zeigt dann die Werte der Felder sowie die lokalen Variablen an.

Mittels "Step Over" (F10) wird das Programm bis nach der nŠchsten Codezeile ausgefŸhrt. "Step Into" (F11) arbeitet auch etwaige Methodenaufrufe nur Zeile fŸr Zeile ab. "Continue" (F5) setzt hingegen die normale AusfŸhrung des Programmes fort.

Kapitel 4. Unit-Tests und TDD (xUnit)

Um Fehler in der zu entwickelnden Software zu vermeiden, ist die DurchfŸhrung von zahlreichen Tests unerlŠsslich. In .NET verwenden wir hŠufig das xUnit-Framework fŸr Unit-Tests.

4.1. Test-Driven Development (TDD)

TDD ist eine Methodik, bei der Tests vor dem eigentlichen Code geschrieben werden. Dies folgt dem Red-Green-Refactor Zyklus: 1. Red: Schreiben Sie einen Test fŸr eine neue Funktion. Da die Funktion noch nicht existiert, schlŠgt der Test fehl. 2. Green: Schreiben Sie gerade so viel Code, dass der Test besteht. 3. Refactor: Verbessern Sie den Code (Lesbarkeit, Performance), wŠhrend Sie sicherstellen, dass alle Tests weiterhin bestehen.

4.2. Testprojekt einrichten

Ein Testprojekt wird separat zur eigentlichen Anwendung erstellt:

```
# Erstellen eines xUnit Testprojekts
dotnet new xunit -n MeinProjekt.Tests
cd MeinProjekt.Tests

# Referenz auf das zu testende Projekt hinzufŸgen
dotnet add reference ../MeinProjekt/MeinProjekt.csproj
```

4.3. Facts und Theories

In xUnit gibt es zwei Hauptarten von Tests:

1. [Fact]: Ein Test, der immer wahr sein muss. Er hat keine Parameter.
2. [Theory]: Ein Test, der fŸr verschiedene DatensŠtze geprŸft wird. Mittels `[InlineData]` kšnnen Parameter Ÿbergeben werden.

```
using Xunit;

public class PersonTests
{
Ê   // Fact: Ein einzelner Testfall
Ê   [Fact]
Ê   public void Konstruktor_MitGueltigenWerten_ErzeugtPerson()
Ê   {
Ê       // Arrange
Ê       var person = new Person("Max Maier", true, 180, 80);

Ê       // Act & Assert
Ê       Assert.Equal("Max Maier", person.Name);
Ê   }

Ê   // Theory: Testen mit mehreren DatensŠtzen
Ê   [Theory]
Ê   [InlineData(180, 80, 24.69)]
Ê   [InlineData(170, 50, 17.30)]
Ê   [InlineData(190, 100, 27.70)]
Ê   public void BerechneBmi_BerechnetKorrekt(int groesse, int gewicht, double erwarteterBmi)
Ê   {
Ê       // Arrange
Ê       var person = new Person("Test", true, groesse, gewicht);
```

```
Ê        // Act
Ê        double bmi = person.BerechneBmi();

Ê        // Assert
Ê        Assert.Equal(erwarteterBmi, bmi, 2); // 2 Nachkommastellen Toleranz
Ê    }

Ê    [Fact]
Ê    public void Konstruktor_MitUngueltigerGroesse_WirftException()
Ê    {
Ê        // Act & Assert
Ê        Assert.Throws<ArgumentOutOfRangeException>(() =>
Ê            new Person("Max Maier", true, 50, 80));
Ê    }
}
```

4.4. Setup und Lifecycle

In xUnit wird fŸr jeden einzelnen Test eine neue Instanz der Testklasse erzeugt. * Setup: Gemeinsamer Vorbereitungscode (z.B. Erzeugen von Testdaten) wird einfach in den Konstruktor der Testklasse geschrieben. * Cleanup: Falls Ressourcen freigegeben werden mŸssen (z.B. Dateien oder Datenbankverbindungen), implementiert die Testklasse das Interface `IDisposable` und nutzt die `Dispose()`-Methode.

4.5. Wichtige Assertions

Die Klasse `Assert` bietet zahlreiche Methoden zur PrŸfung von Ergebnissen:

Methode	Bedeutung
`Assert.Equal(exp, act)`	PrŸft auf Gleichheit (Werte oder Strings).
`Assert.NotEqual(exp, act)`	PrŸft auf Ungleichheit.
`Assert.Null(obj)` / `Assert.NotNull(obj)`	PrŸft, ob eine Referenz null (oder nicht null) ist.
`Assert.True(cond)` / `Assert.False(cond)`	PrŸft eine logische Bedingung.
`Assert.Contains(sub, str)`	PrŸft, ob ein Teilstring oder ein Element in einer Liste vorhanden ist.
`Assert.Throws<T>(É)`	PrŸft, ob der Code eine Exception vom Typ T wirft.
`Assert.InRange(val, low, high)`	PrŸft, ob ein Wert innerhalb eines Bereichs liegt.

4.6. Tests ausfŸhren

Tests kšnnen auf verschiedene Arten gestartet werden:

1. Kommandozeile: Geben Sie `dotnet test` im Projektverzeichnis ein.
2. JetBrains Rider: Nutzen Sie das "Unit Tests" Fenster (Strg+Alt+U) oder klicken Sie auf die Symbole neben den Testmethoden im Editor.
3. VS Code: †ber den "Testing" View in der Seitenleiste (Symbol mit dem Reagenzglas) oder via `C# Dev Kit`.

Ziel ist es, eine mšglichst hohe Testabdeckung des Codes zu erreichen. Das AusfŸhren dieser Tests liefert dann quasi per Knopfdruck als Ergebnis, ob (nach etwaigen €nderungen) Fehler

im Code vorhanden sind.

Kapitel 5. Klassen, Objekte und Methoden

Klassen sind ein grundlegendes Konzept moderner Programmiersprachen. Sie enthalten den wesentlichen Programmcode, ganze Programme entstehen durch ein Zusammenspiel mehrerer Klassen.

Klassen kšnnen unterschiedlichste Aufgaben in Programmen Ÿbernehmen. Wir wollen aber zunŠchst eines der wichtigsten Einsatzgebiete betrachten. HŠufig modellieren Klassen EntitŠten aus der realen Welt. Eine Anwendung kšnnte beispielsweise dazu dienen *Studenten* einer Schule zu verwalten. Dazu erstellen wir zunŠchst eine Klasse die einen Studenten modelliert.

```
public class Student
{
Ê   private string _name;
Ê   private int _geburtsjahr;
}
```

Dieser Student hat nur zwei Eigenschaften (auch Attribute oder Felder genannt), nŠmlich einen Namen und ein Geburtsjahr. Der Name (`_name`) ist dabei eine Text-Zeichenkette (`string`), das Geburtsjahr (`_geburtsjahr`) ist eine ganze Zahl (`int`). Die Angabe `private` legt fest, dass diese Elemente au§erhalb der Klasse nicht sichtbar sind.

Es ist unbedingt zu beachten, dass Gro§- und Klein-Schreibung eine Rolle spielen. Der Name der Klasse (in diesem Fall `Student`) beginnt stets mit einem Gro§buchstaben (PascalCase). Die Felder (`_name` und `_geburtsjahr`) beginnen stets mit einem Unterstrich und Kleinbuchstaben (camelCase mit Unterstrich-PrŠfix).

Die Strichpunkte am Ende einer Zeile sind hier ebenso wesentlich. Sie beenden eine sogenannte *Anweisung*. Werden sie weggelassen, kann das Programm nicht Ÿbersetzt werden.

Beachten Sie weiters die Formatierung dieses CodestŸckes. Die beiden Zeilen zu den Feldern sind eingerŸckt (4 Leerzeichen), die schlie§ende geschwungene Klammer ist wieder ganz links gesetzt. Dies ist eine wichtige Konvention, die unbedingt einzuhalten ist.

Wenden wir uns nun der Bedeutung von Klassen zu. Eine Klasse modelliert eine EntitŠt aus der realen Welt, wie zum Beispiel Studenten, Personen, Aktien, Wohnungen, Fahrzeuge, usw. Konkret wird festgelegt, welche Daten diese EntitŠten enthalten sollen, in unserem Beispiel einen Namen und ein Geburtsjahr. Die Klasse selbst dient im Programm als *Vorlage* fŸr konkrete Daten. Konkrete Daten zur Klasse `Student` mit `Name` und `Geburtsjahr` kšnnen beispielsweise sein: Peter (1981), Sandra (1985), Ercan (1991), Sarah (1994); derartige konkrete AusprŠgungen einer Klasse werden Objekte genannt. Erstellt man also einen konkreten Studenten, so erzeugt man im Programm ein Objekt oder eine Instanz einer Klasse. Die Klasse stellt also einen "Bauplan" dar, wŠhrend Objekte oder Instanzen dann konkrete AusprŠgungen desselben sind.

Wir werden in weiterer Folge kennenlernen wie auf die Eigenschaften zugegriffen werden kann (Abfragen von Informationen eines Objektes), und wie diese verŠndert werden kšnnen. Weiters werden wir die Erzeugung von Objekten nŠher betrachten. Am Ende des Abschnittes werden Sie eine wichtige weitere Eigenschaft von Klassen kennenlernen. Sie kšnnen nŠmlich

nicht nur Daten enthalten, sondern auch *Methoden*. Dies sind ProgrammstŸcke, die auf den Daten Berechnungen vornehmen, oder auch VerŠnderungen vornehmen. Klassen organisieren also das Zusammenspiel von Daten und Programmlogik, Programme selbst entstehen durch die Interaktion von mehreren Objekten.

5.1. Properties (Eigenschaften)

In C# verwenden wir zur Kapselung von Daten nicht direkt Get- und Set-Methoden wie in Java, sondern sogenannte Properties (Eigenschaften). Dies ist eine elegantere und kompaktere Syntax.

5.1.1. Getter-Properties (read-only)

Eine read-only Property ermšglicht nur das Lesen eines Wertes:

```
public class Student
{
Ê   private string _name;
Ê   private int _geburtsjahr;

Ê   public string Name => _name;

Ê   public int Geburtsjahr => _geburtsjahr;
}
```

Die Syntax `%` ist eine Expression-bodied Property. Sie ist Šquivalent zu:

```
public string Name
{
Ê   get { return _name; }
}
```

5.1.2. Setter-Properties

Properties kšnnen auch Schreibzugriff ermšglichen. Wir nutzen dabei das Fail-Fast-Pattern fŸr Validierungen:

```
public class Student
{
Ê   private string _name;
Ê   private int _geburtsjahr;

Ê   public string Name
Ê   {
Ê       get => _name;
Ê       set
Ê       {
Ê           if (string.IsNullOrWhiteSpace(value))
Ê               throw new ArgumentException("Name darf nicht leer sein", nameof(value));
Ê           _name = value;
Ê       }
Ê   }

Ê   public int Geburtsjahr
Ê   {
Ê       get => _geburtsjahr;
Ê       set
Ê       {
```

```
Ê            if (value < 1900 || value > DateTime.Now.Year)
Ê                throw new ArgumentOutOfRangeException(nameof(value),
Ê                    "Geburtsjahr muss zwischen 1900 und dem aktuellen Jahr liegen");
Ê            _geburtsjahr = value;
Ê        }
Ê    }
}
```

Das SchlŸsselwort `value` reprŠsentiert den Ÿbergebenen Wert im Setter.

5.2. Auto-Properties

FŸr einfache FŠlle ohne Validierungslogik kšnnen Auto-Properties verwendet werden:

```
public class Student
{
Ê    public string Name { get; set; }
Ê    public int Geburtsjahr { get; set; }
}
```

Der Compiler erzeugt automatisch ein privates Feld im Hintergrund. FŸr šffentliche APIs mit Validierungsanforderungen sollten jedoch explizite Properties bevorzugt werden.

5.3. Kommentare

Fallweise mšchte man im Programm ergŠnzende Kommentare aufnehmen. Diese stehen zwar im Quelltext, werden aber vom Compiler ignoriert.

```
/*
Dies ist ein mehrzeiliges Kommentar. Am Beginn einer Klasse kann es zum Beispiel
Informationen zum Zweck der Klasse, Autor, Erstellungsdatum etc. enthalten.
*/
public class Student
{
Ê    // dies ist ein einzeiliger Kommentar

Ê    private string _name;
Ê    private int _geburtsjahr;

Ê    public string Name => _name; // Kommentare kšnnen auch nach Code stehen

Ê    /*
Ê    public int Geburtsjahr => _geburtsjahr;
Ê    */
}
```

5.4. Konstruktoren

Der Konstruktor legt fest, *wie* ein Objekt erzeugt wird, also welche Werte dabei den Feldern zugewiesen werden kšnnen. Wir verwenden den Konstruktor, um die Validierungslogik aus den Properties wiederzuverwenden.

```
public class Student
{
Ê    private string _name;
Ê    private int _geburtsjahr;

Ê    public Student()
```

```
Ê   {
Ê       _name = "n/a";
Ê       _geburtsjahr = 1970;
Ê   }

Ê   public Student(string name, int geburtsjahr)
Ê   {
Ê       if (string.IsNullOrWhiteSpace(name))
Ê           throw new ArgumentException("Name darf nicht leer sein", nameof(name));
Ê       if (geburtsjahr < 1900 || geburtsjahr > DateTime.Now.Year)
Ê           throw new ArgumentOutOfRangeException(nameof(geburtsjahr));

Ê       _name = name;
Ê       _geburtsjahr = geburtsjahr;
Ê   }

Ê   // Properties hier ausgelassen
}
```

Konstruktoren kšnnen ebenso wie Methoden Ÿberladen werden - eine Klasse kann mehrere Konstruktoren mit unterschiedlichen Parameterlisten haben.

5.5. Methoden

Methoden sind Bestandteile von Klassen, die bestimmte Aufgaben durchfŸhren, wie zum Beispiel €nderungen von Werten, DurchfŸhrung von Berechnungen, etc. Sie haben schon spezielle Methoden kennengelernt, nŠmlich Konstruktoren. Allgemein bestehen Methoden aus einem *Methodennamen*, einem *RŸckgabewert*, *Parametern*, sowie einem *Methodenrumpf*, der den eigentlichen Code enthŠlt. *Methodennamen*, *RŸckgabewert* und *Parameter* (sowie der Zugriffs-Modifier) bilden den *Methodenkopf*. Wir betrachten folgende Methode zur Berechnung des aktuellen Alters einer Person:

```
public class Student
{
Ê   private string _name;
Ê   private int _geburtsjahr;

Ê   public int BerechneAlter(int aktuellesJahr)
Ê   {
Ê       if (aktuellesJahr < _geburtsjahr)
Ê           throw new ArgumentException("Aktuelles Jahr muss nach Geburtsjahr liegen");

Ê       return aktuellesJahr - _geburtsjahr;
Ê   }
}
```

Die Methode `BerechneAlter` bekommt als Parameter das aktuelle Jahr Ÿbergeben. Das Alter der Person ergibt sich dann aus der Differenz von `aktuellesJahr` und `_geburtsjahr`. Im Methodenkopf wurde der Typ des RŸckgabewertes mit `int` festgelegt. Die Berechnung dieses Wertes erfolgt dann im Methodenrumpf. Das SchlŸsselwort `return` gibt an, dass nun der angesprochene RŸckgabewert retourniert wird.

5.5.1. Ausgabe, Console.WriteLine

Mittels `Console.WriteLine` kšnnen Ausgaben auf die Konsole gemacht werden. Berechnungen und Ergebnisse kšnnen somit dem Benutzer (Anwender) des Programmes angezeigt werden.

Zur Ausgabe von Objekten erstellen wir jeweils eigene Methoden oder Ÿberschreiben `ToString()`.

```
public class Student
{
Ê   private string _name;
Ê   private int _geburtsjahr;

Ê   public override string ToString()
Ê   {
Ê       return $"Name: {_name} (Geburtsjahr: {_geburtsjahr})";
Ê   }

Ê   public void Ausgeben()
Ê   {
Ê       Console.WriteLine(this);
Ê   }
}
```

Das `$"É"` ist eine interpolated string literal, die es ermšglicht, Variablen direkt in Strings einzubetten.

5.6. Signatur von Methoden

Die Signatur einer Methode ist gegeben durch den Methodennamen, sowie Typen und Reihenfolge der Parameter. In einer Klasse dŸrfen nicht mehrere Methoden mit der selben Signatur vorkommen. Mehrere Methoden mit gleichem Namen sind sehr wohl zulŠssig, da sie eindeutig durch die Parameter unterschieden werden kšnnen. Man nennt dies auch *†berladen* von Methoden.

Wichtig: Der RŸckgabewert gehšrt *nicht* zur Signatur!

```
public class SignaturDemo
{
Ê   public int Foo(int a, int b) { return a + b; }

Ê   public int Foo(int a) { return a * 2; } // ZulŠssig: andere Parameteranzahl

Ê   public double Foo(double a, double b) { return a + b; } // ZulŠssig: andere Parametertypen

Ê   /*
Ê   public double Foo(int x) { return x * 1.0; }
Ê   UNZUL€SSIG: Es existiert bereits eine Methode Foo(int).
Ê   Der unterschiedliche RŸckgabewert (double vs int) reicht zur Unterscheidung nicht aus!
Ê   */
}

public class Berechnung
{
Ê   public int Summe(int a, int b)
Ê   {
Ê       return a + b;
Ê   }

Ê   public double Summe(double a, double b)
Ê   {
Ê       return a + b;
Ê   }
```

```
}
```

Hier ist es sinnvoll, die Methode zur Summenbildung von `int`-Parametern sowie `double`-Parametern gleich zu benennen, also zu Ÿberladen.

5.7. Parameter-PrŸfungen (Fail-Fast)

Wir betrachten in diesem Abschnitt die Klasse `Person` mit Fail-Fast-Validierung:

```
public class Person
{
Ê   private string _name;
Ê   private bool _maennlich;
Ê   private int _groesse;
Ê   private int _gewicht;

Ê   public Person(string name, bool maennlich, int groesse, int gewicht)
Ê   {
Ê       if (string.IsNullOrWhiteSpace(name))
Ê           throw new ArgumentException("Name darf nicht leer sein", nameof(name));
Ê       if (groesse < 110 || groesse > 220)
Ê           throw new ArgumentOutOfRangeException(nameof(groesse),
Ê               "Grš§e muss zwischen 110 und 220 cm liegen");
Ê       if (gewicht < 30 || gewicht > 150)
Ê           throw new ArgumentOutOfRangeException(nameof(gewicht),
Ê               "Gewicht muss zwischen 30 und 150 kg liegen");

Ê       _name = name;
Ê       _maennlich = maennlich;
Ê       _groesse = groesse;
Ê       _gewicht = gewicht;
Ê   }

Ê   public string Name => _name;
Ê   public bool Maennlich => _maennlich;
Ê   public int Groesse => _groesse;
Ê   public int Gewicht => _gewicht;

Ê   public override string ToString()
Ê   {
Ê       string geschlecht = _maennlich ? "m" : "f";
Ê       return $"Name: {_name} ({geschlecht}), Grš§e: {_groesse}, Gewicht: {_gewicht}";
Ê   }
}
```

Im Gegensatz zum Happy-Path-Pattern werden hier bei ungŸltigen Parametern sofort Exceptions geworfen. Der Aufrufer muss sich um die Fehlerbehandlung kŸmmern, was zu robusterem Code fŸhrt.

5.8. Lokale Variablen, Berechnungen und Typkonvertierung

Wir betrachten nun die folgende Aufgabenstellung: Berechnung des BMI (Body-Mass-Index).

```
public class Person
{
Ê   // ... Felder und Properties wie oben ...

Ê   public double BerechneBmi()
Ê   {
```

```
Ê       double groesseMeter = _groesse / 100.0;
Ê       return _gewicht / (groesseMeter * groesseMeter);
Ê   }

Ê   public void BmiAusgeben()
Ê   {
Ê       double bmi = BerechneBmi();
Ê       string bmiTyp;

Ê       if (_maennlich)
Ê       {
Ê           if (bmi < 20)
Ê               bmiTyp = "Untergewicht";
Ê           else if (bmi <= 25)
Ê               bmiTyp = "Normalgewicht";
Ê           else
Ê               bmiTyp = "†bergewicht";
Ê       }
Ê       else
Ê       {
Ê           if (bmi < 19)
Ê               bmiTyp = "Untergewicht";
Ê           else if (bmi <= 24)
Ê               bmiTyp = "Normalgewicht";
Ê           else
Ê               bmiTyp = "†bergewicht";
Ê       }

Ê       Console.WriteLine($"BMI: {bmi:F2} - {bmiTyp}");
Ê   }
}
```

In der Methode `BerechneBmi` wird zunŠchst die Grš§e von Zentimeter in Meter umgerechnet. Beachten Sie, dass wir `100.0` verwenden, um eine Flie§kommadivision zu erzwingen. Die Formatangabe `F2` in `Console.WriteLine` gibt an, dass der Wert mit 2 Nachkommastellen formatiert werden soll.

Kapitel 6. Variablen, Datentypen und Methoden

Im letzten Abschnitt wurden bereits die Datentypen `string` und `int` verwendet. Diese fanden Anwendung bei *Feldern* und *Parametern*. Der Oberbegriff dafŸr lautet Variablen, wir werden spŠter noch eine weitere Art von Variablen, nŠmlich *lokale Variablen* kennen lernen.

Der Datentyp legt fest, welche Art von Werten bei einer Variablen hinterlegt sind. Wir haben bereits `int` als Datentyp fŸr ganze Zahlen kennengelernt, und `string` als Datentyp fŸr Zeichenketten. Wir betrachten zunŠchst die sogenannten *Werttypen* genauer.

6.1. Werttypen

Die Werttypen (value types) sind in der Programmiersprache C# "fest eingebaut". Die wichtigsten Werttypen umfassen:

Typ	Bereich	Verwendung
`byte`	0 bis 255	Kleine positive Ganzzahlen
`int`	-2.147.483.648 bis 2.147.483.647	Ganze Zahlen (Standard)
`long`	-9.223.372.036.854.775.808 bis 9.223.372.036.854.775.807	Sehr gro§e Ganzzahlen
`float`	±1,5 ! 10^-45 bis ±3,4 ! 10^38	Kommazahlen (prŠzise)
`double`	±5,0 ! 10^-324 bis ±1,7 ! 10^308	Kommazahlen (Standard)
`decimal`	±1,0 ! 10^-28 bis ±7,9 ! 10^28	Finanz- und monetŠre Berechnungen
`bool`	true oder false	Logische Wahrheitswerte
`char`	Unicode-Zeichen	Einzelnes Zeichen
`DateTime`	01.01.0001 bis 31.12.9999	Datum und Uhrzeit

FŸr ganze Zahlen ist meist `int` die beste Wahl, nur wenn extrem gro§e Zahlen dargestellt werden sollen muss `long` verwendet werden. FŸr Kommazahlen wird meist `double` verwendet. FŸr finanzielle Berechnungen ist `decimal` zu bevorzugen, da es keine Rundungsfehler hat.

6.2. Referenztypen

Im vorigen Kapitel haben wir die Klasse `Student` entwickelt, und damit einen eigenen *Datentyp* erstellt! *Alle* Klassen definieren einen Datentyp, im Gegensatz zu den Werttypen, die ja ein integraler Bestandteil der Programmiersprache sind, handelt es sich hierbei im einen Referenztyp. Auch beim schon verwendeten Typ `string` handelt es sich um einen derartigen Referenztyp.

Ein wesentlicher Unterschied von Referenztypen zu Werttypen ist, dass sie nicht den eigentlichen *Wert* selbst enthalten, sondern eine *Referenz* (in andren Worten: einen Verweis)

auf den Speicherort, an dem die Daten tatsŠchlich gespeichert sind.

Daraus ergeben sich die folgenden Konsequenzen fŸr den Umgang mit Referenztypen:

- Der Verweis kann auch "ins Leere" gehen, also auf nichts Eigentliches verweisen. Der Wert der Objektreferenz ist dann `null`. Ist der Wert einer Objektreferenz `*null*`, so kšnnen darauf keine Methoden aufgerufen werden. Dies wŸrde zu einer `NullReferenceException` fŸhren!
- Umgekehrt ist es mšglich, dass auf ein und den selben Inhalt (im Speicher) mehrere Referenzen existieren. Es kann also eine Variable `stud1` auf den selben Studenten wie die Variable `stud2` verweisen!

6.3. Methoden (Fortsetzung)

Wir haben schon spezielle Methoden kennengelernt, nŠmlich Properties und Konstruktoren. Methoden sind also zusammengehšrige Code-StŸcke, die Teil einer Klasse sind. Eine Methode setzt sich aus dem *Methodenkopf* und dem *Methodenrumpf* zusammen. Der Methodenkopf enthŠlt einen Zugriffs-Modifier (`private` oder `public`), einen RŸckgabewert, einen Methoden-Namen, sowie eine mšglicherweise auch leere Parameterliste.

Der Zugriffsmodifier steuert die *"Sichtbarkeit"* der Methode. Alle `private` gesetzten Elemente sind au§erhalb der Klasse *nicht* sichtbar, d.h. sie kšnnen von dort aus nicht aufgerufen werden. Hingegen bedeutet `public`, dass das Element Ÿberall sichtbar ist.

Der RŸckgabewert einer Methode ist entweder ein zurŸckgegebener Wert, oder `void` fŸr Methoden, die nichts zurŸckgeben.

In den runden Klammern sind die Parameter einer Methode angegeben. StandardmŠ§ig werden in C# alle Parameter als Kopie Ÿbergeben (Call-by-Value):

- Bei Werttypen (z. B. `int`, `double`, `bool`) wird der eigentliche Wert kopiert. €nderungen am Parameter innerhalb der Methode haben keine Auswirkungen auf die ursprŸngliche Variable des Aufrufers.
- Bei Referenztypen (z. B. Klassen, Strings) wird die Referenz (der Verweis) kopiert. Da die kopierte Referenz auf dasselbe Objekt im Speicher zeigt, wirken sich €nderungen am Zustand des Objekts (z. B. €ndern eines Feldes) direkt aus. Die Variable des Aufrufers kann jedoch nicht innerhalb der Methode auf ein všllig neues Objekt umgebogen werden.

```
public class Student
{
Ê   private string _name;
Ê   private int _geburtsjahr;

Ê   public int BerechneAlter(int aktuellesJahr)
Ê   {
Ê       // Parameter 'aktuellesJahr' ist eine Kopie des Wertes
Ê       return aktuellesJahr - _geburtsjahr;
Ê   }
}
```

6.4. ParameterŸbergabe mit `ref` und `out`

Mšchte man eine Methode in die Lage versetzen, die Variable des Aufrufers direkt zu verŠndern (Call-by-Reference), nutzt man die SchlŸsselwšrter `ref` oder `out`.

Das SchlŸsselwort `ref` Wird ein Parameter mit `ref` markiert, wird keine Kopie erstellt, sondern eine Referenz auf die Variable selbst Ÿbergeben. Die Variable muss vor dem Aufruf initialisiert sein.

```
public void Verdopple(ref int zahl)
{
Ê   zahl = zahl * 2; // €ndert die Variable des Aufrufers direkt
}

// Aufruf:
int meinWert = 10;
Verdopple(ref meinWert);
// meinWert ist nun 20
```

Das SchlŸsselwort `out` `out` funktioniert Šhnlich wie `ref`, wird aber verwendet, wenn eine Methode einen Wert "hinausgeben" soll. Die Variable muss vor dem Aufruf nicht initialisiert sein, aber die Methode muss ihr vor dem Ende einen Wert zuweisen.

```
public void BerechneWerte(int a, int b, out int summe, out int produkt)
{
Ê   summe = a + b;
Ê   produkt = a * b;
}

// Aufruf:
BerechneWerte(10, 5, out int s, out int p);
// s = 15, p = 50
```

6.5. Wann ist die Verwendung von `ref` und `out` sinnvoll?

Die Verwendung von `ref` und `out` sollte sparsam erfolgen, da sie den Code schwerer lesbar machen kšnnen. Es gibt jedoch klare AnwendungsfŠlle:

- ¥ Das Try-Pattern (fŸr `out`): Dies ist der hŠufigste Fall in der .NET-Entwicklung. Viele Methoden geben einen `bool` zurŸck, um Erfolg zu signalisieren, und den eigentlichen Wert Ÿber `out`. Das bekannteste Beispiel ist `int.TryParse`.
- ¥ RŸckgabe mehrerer Werte: Wenn eine Methode mehr als ein Ergebnis liefern muss. Obwohl man hier heutzutage oft Tuples bevorzugt, ist `out` eine bewŠhrte Methode.
- ¥ Performance (fŸr `ref`): Bei extrem gro§en Werttypen (structs), um das Kopieren des gesamten Objekts zu vermeiden. Da wir in der Regel mit Klassen (`class`) arbeiten, die ohnehin als Referenz Ÿbergeben werden, ist dies selten notwendig.

Best Practice: Nutzen Sie fŸr normale Berechnungen immer den RŸckgabewert (`return`). Verwenden Sie `out` primŠr fŸr das Try-Pattern und vermeiden Sie `ref`, sofern es keine zwingenden technischen GrŸnde gibt.

Kapitel 7. Objekt-Beziehungen

Oftmals stehen Objekte in Beziehung zueinander. Ein hŠufiger Fall ist, dass in Objekten einer Klasse mehrere Instanzen einer anderen Klasse verwaltet werden. Im vorigen Abschnitt wurde die Klasse `Person` entwickelt. In diesem Abschnitt wird eine Klasse `Auto` entwickelt, in das bis zu drei Personen "einsteigen" kšnnen.

Die PlŠtze im Auto werden durch Felder, konkret durch Referenzen auf Objekte vom Typ `Person` umgesetzt. Steigt eine konkrete Person in das Auto ein, so wird im Objekt des Typs Auto eine Objektreferenz auf das Objekt der Person gesetzt.

```
public class Auto
{
Ê   private string _name;
Ê   private int _eigengewicht;

Ê   private Person _fahrer;
Ê   private Person _beifahrer;
Ê   private Person _rueckbank;

Ê   public Auto(string name, int eigengewicht)
Ê   {
Ê       if (string.IsNullOrWhiteSpace(name))
Ê           throw new ArgumentException("Name darf nicht leer sein", nameof(name));
Ê       if (eigengewicht < 600 || eigengewicht > 3000)
Ê           throw new ArgumentOutOfRangeException(nameof(eigengewicht));

Ê       _name = name;
Ê       _eigengewicht = eigengewicht;
Ê   }

Ê   public string Name => _name;
Ê   public int Eigengewicht => _eigengewicht;

Ê   public void Einsteigen(Person person)
Ê   {
Ê       if (person == null)
Ê           throw new ArgumentNullException(nameof(person));

Ê       if (_fahrer == null)
Ê           _fahrer = person;
Ê       else if (_beifahrer == null)
Ê           _beifahrer = person;
Ê       else if (_rueckbank == null)
Ê           _rueckbank = person;
Ê       else
Ê           throw new InvalidOperationException("Das Auto ist voll!");
Ê   }

Ê   public void Aussteigen(Person person)
Ê   {
Ê       if (person == null)
Ê           throw new ArgumentNullException(nameof(person));

Ê       if (_fahrer == person)
Ê           _fahrer = null;
Ê       else if (_beifahrer == person)
Ê           _beifahrer = null;
Ê       else if (_rueckbank == person)
Ê           _rueckbank = null;
```

```
Ê       else
Ê           throw new InvalidOperationException("Die Person sitzt nicht im Auto.");
Ê   }

Ê   public void Aussteigen(string name)
Ê   {
Ê       if (string.IsNullOrWhiteSpace(name))
Ê           throw new ArgumentException("Name darf nicht leer sein", nameof(name));

Ê       // In C# kšnnen Strings einfach mit == verglichen werden
Ê       if (_fahrer != null && _fahrer.Name == name)
Ê           _fahrer = null;
Ê       else if (_beifahrer != null && _beifahrer.Name == name)
Ê           _beifahrer = null;
Ê       else if (_rueckbank != null && _rueckbank.Name == name)
Ê           _rueckbank = null;
Ê       else
Ê           throw new InvalidOperationException($"Person '{name}' nicht im Auto gefunden.");
Ê   }

Ê   public int Gesamtgewicht()
Ê   {
Ê       int gewicht = _eigengewicht;

Ê       if (_fahrer != null)
Ê           gewicht += _fahrer.Gewicht;
Ê       if (_beifahrer != null)
Ê           gewicht += _beifahrer.Gewicht;
Ê       if (_rueckbank != null)
Ê           gewicht += _rueckbank.Gewicht;

Ê       return gewicht;
Ê   }

Ê   public override string ToString()
Ê   {
Ê       var sb = new System.Text.StringBuilder();
Ê       sb.AppendLine($"Auto: {_name}, Eigengewicht: {_eigengewicht}");
Ê       sb.AppendLine(new string('-', 50));
Ê       sb.AppendLine($"Fahrer: {_fahrer?.ToString() ?? "--frei--"}");
Ê       sb.AppendLine($"Beifahrer: {_beifahrer?.ToString() ?? "--frei--"}");
Ê       sb.AppendLine($"RŸckbank: {_rueckbank?.ToString() ?? "--frei--"}");
Ê       sb.AppendLine(new string('-', 50));

Ê       return sb.ToString();
Ê   }
}
```

Die wesentlichen †berlegungen zu derartigen Aufgaben finden sich in der Methode `Einsteigen`. Diese Methode hat einen Parameter vom Typ `Person` mittels dem die Objektreferenz auf die einsteigende Person Ÿbergeben wird. Die zugrundeliegende Logik ist: die Person nimmt den ersten freien Platz. Die Methode verwendet das Fail-Fast-Pattern und wirft eine Exception wenn das Auto voll ist.

Beachten Sie im `ToString()`-Method die Verwendung des *null-conditional operators* (`?.`) und des *null-coalescing operators* (`??`). Diese ermšglichen eine kompakte und sichere Handhabung von potentiell null-Werten.

Kapitel 8. Strings

Der Typ `string` ist ein integraler Bestandteil von C# und enthŠlt zahlreiche nŸtzliche Methoden zur Bearbeitung und Analyse von Zeichenketten. Strings sind in C# unverŠnderlich (immutable) - jede Modifikation erzeugt einen neuen String.

Wichtige Methoden der Klasse `string`:

- `char this[int index]`: Gibt das Zeichen an der Stelle `index` zurŸck (erste Stelle hat Index 0!)
- `int IndexOf(string str)`: Gibt die Stelle des ersten Vorkommens des Ÿbergebenen Strings zurŸck
- `bool IsNullOrEmpty(string)`: Statische Methode, prŸft ob der String null oder leer ist
- `bool IsNullOrWhiteSpace(string)`: Statische Methode, prŸft auf null, leer oder nur Leerzeichen
- `int Length`: Gibt die LŠnge des Strings zurŸck
- `string ToLower()`: Retourniert den String in Kleinbuchstaben
- `string ToUpper()`: Retourniert den String in Gro§buchstaben
- `string Trim()`: Entfernt Leerzeichen am Anfang und Ende
- `string[] Split(char[] separator)`: Teilt den String an den angegebenen Trennzeichen
- `bool StartsWith(string)`: PrŸft ob der String mit dem angegebenen Text beginnt
- `bool EndsWith(string)`: PrŸft ob der String mit dem angegebenen Text endet

```
public class Person
{
Ê   private string _vorname;
Ê   private string _nachname;

Ê   public Person(string vorname, string nachname)
Ê   {
Ê       if (string.IsNullOrWhiteSpace(vorname))
Ê           throw new ArgumentException("Vorname darf nicht leer sein", nameof(vorname));
Ê       if (string.IsNullOrWhiteSpace(nachname))
Ê           throw new ArgumentException("Nachname darf nicht leer sein", nameof(nachname));

Ê       _vorname = vorname;
Ê       _nachname = nachname;
Ê   }

Ê   public string VollerName => $"{_vorname} {_nachname}";

Ê   public int LaengeVollerName => VollerName.Length;

Ê   public string Initialen
Ê   {
Ê       get
Ê       {
Ê           if (_vorname.Length == 0 || _nachname.Length == 0)
Ê               throw new InvalidOperationException("Vor- oder Nachname ist leer");

Ê           return $"{_vorname[0]}. {_nachname[0]}.";
Ê       }
Ê   }

Ê   public string Kuerzel
Ê   {
```

```
Ê       get
Ê       {
Ê           if (_nachname.Length < 2 || _vorname.Length < 1)
Ê               throw new InvalidOperationException("Name zu kurz fŸr KŸrzel");

Ê           return $"{_nachname.Substring(0, 2).ToUpper()}{_vorname[0]}";
Ê       }
Ê   }
}
```

8.1. String-Vergleich

Mšchte man zwei Strings auf Inhaltsgleichheit ŸberprŸfen, sollte die `==`- oder `!=`-Operatoren verwendet werden (im Gegensatz zu Java, wo `equals()` benštigt wird). In C# ist der `==`-Operator fŸr Strings Ÿberladen und vergleicht den Inhalt:

```
string a = "xyz";
string b = "xy" + "z";

// Beide Vergleiche funktionieren korrekt in C#:
Console.WriteLine(a == b);        // true (Inhaltsvergleich)
Console.WriteLine(a.Equals(b));   // true (expliziter Methodenaufruf)
```

ï¿½berprï¿½ft ob es das selbe Ob
gleicht das a dem b,
verschiedene Objekte aber gleichen sich

beim Vergleichen von Strings immer Equals nehmen -Graf

8.2. StringBuilder

Da Strings in C# unverŠnderlich (immutable) sind, erzeugt jede €nderung (z.B. mit `+`) ein neues String-Objekt im Speicher. Bei vielen €nderungen in einer Schleife kann dies die Performance beeintrŠchtigen. In solchen FŠllen verwendet man den `StringBuilder`.

```
using System.Text;

var sb = new StringBuilder();
sb.Append("Start");
for (int i = 0; i < 100; i++)
{
Ê   sb.Append(i);
Ê   sb.Append(", ");
}
string ergebnis = sb.ToString();
```

strings sind Objekte keine Basis Operator!

strings unterstï¿½tzt auch grï¿½ï¿½er gleich usw. Oper

mit Strings kann man auch eine alphabetische Sortierung machen

Kapitel 9. Schleifen

Schleifen sind zentrale Sprachkonstrukte in allen Programmiersprachen. Sie ermšglichen es bestimmte Aufgaben mehrmals durchzufŸhren.

Kopf und Fuï¿½gesteuerte Arte

9.1. `while`-Schleife

```
public void ZeichneLinie(int laenge)
{
Ê   if (laenge < 0)
Ê       throw new ArgumentOutOfRangeException(nameof(laenge));

Ê   int zaehler = 0;
Ê   while (zaehler < laenge)
Ê   {
Ê       Console.Write("-");
Ê       zaehler++;
Ê   }
Ê   Console.WriteLine();
}
```

9.2. `do-while`-Schleife

Fuï¿½gesteurte Schleif

```
public void ZahlenAusgeben()
{
Ê   int i = 0;
Ê   do
Ê   {
Ê       Console.WriteLine(i);
Ê       i++;
Ê   } while (i < 10);
}
```

9.3. `for`-Schleife

```
public void ZahlenAusgeben2()
{
Ê   for (int i = 0; i < 10; i++)
Ê   {
Ê       Console.WriteLine(i);
Ê   }
}
```

9.4. `foreach`-Schleife

FŸr die Iteration Ÿber Collections ist die `foreach`-Schleife besonders elegant:

```
public void NamenAusgeben(List<string> namen)
{
Ê   foreach (string name in namen)
Ê   {
Ê       Console.WriteLine(name);
Ê   }
}
```

9.5. Codebeispiele und Code-Wiederverwendung

Beim Programmieren sollte man stets versuchen, bestimmte Teilaufgaben nicht mehrfach zu implementieren. Das Prinzip der Code-Wiederverwendung besagt, dass schon implementierte CodestŸcke (in Methoden) nach Mšglichkeit Ÿberall dort wiederverwendet werden sollen, wo

sie als Teilaufgabe einer anderen Aufgabe auftreten.

9.5.1. Beispiel: Quadrat mit Diagonale

In diesem Beispiel soll eine Klasse `Quadrat` entwickelt werden, die ein Quadrat mit einer Diagonale von links oben nach rechts unten auf der Konsole ausgeben kann.

```
public class Quadrat
{
Ê   private int _laenge;

Ê   public Quadrat(int laenge)
Ê   {
Ê       if (laenge <= 0 || laenge >= 100)
Ê           throw new ArgumentOutOfRangeException(nameof(laenge), "LŠnge muss zwischen 1 und 99
liegen");

Ê       _laenge = laenge;
Ê   }

Ê   public void Ausgeben()
Ê   {
Ê       for (int zeile = 0; zeile < _laenge; zeile++)
Ê       {
Ê           for (int spalte = 0; spalte < _laenge; spalte++)
Ê           {
Ê               if (zeile == 0 || zeile == _laenge - 1 ||
Ê                   spalte == 0 || spalte == _laenge - 1 ||
Ê                   zeile == spalte)
Ê               {
Ê                   Console.Write("#");
Ê               }
Ê               else
Ê               {
Ê                   Console.Write(" ");
Ê               }
Ê           }
Ê           Console.WriteLine();
Ê       }
Ê   }

Ê   /// <summary>
Ê   /// Zeichnet das Quadrat mit der anderen Diagonale (rechts oben nach links unten)
Ê   /// </summary>
Ê   public void AusgebenInvers()
Ê   {
Ê       for (int zeile = 0; zeile < _laenge; zeile++)
Ê       {
Ê           for (int spalte = 0; spalte < _laenge; spalte++)
Ê           {
Ê               if (zeile == 0 || zeile == _laenge - 1 ||
Ê                   spalte == 0 || spalte == _laenge - 1)
Ê               {
Ê                   Console.Write("#");
Ê               }
Ê               else if (spalte == _laenge - 1 - zeile)
Ê               {
Ê                   Console.Write("*");
Ê               }
Ê               else
Ê               {
```

```
Ê                   Console.Write(" ");
Ê               }
Ê           }
Ê           Console.WriteLine();
Ê       }
Ê   }
}
```

9.5.2. Beispiel: Rechteck mit Code-Wiederverwendung

Das folgende Beispiel demonstriert, wie man eine komplexe Aufgabe (Rechteck zeichnen) in kleinere Teilaufgaben (Linie zeichnen) zerlegt.

```
public class SchleifenDemo
{
Ê   public void ZeichneLinie(int einrueckung, int laenge)
Ê   {
Ê       ZeichneLinie(einrueckung, laenge, '*', '#');
Ê   }

Ê   public void ZeichneLinie(int einrueckung, int laenge, char fuellZeichen, char randZeichen)
Ê   {
Ê       if (einrueckung < 0)
Ê           throw new ArgumentOutOfRangeException(nameof(einrueckung), "EinrŸckung darf nicht
negativ sein");
Ê       if (laenge < 0)
Ê           throw new ArgumentOutOfRangeException(nameof(laenge), "LŠnge darf nicht negativ
sein");

Ê       // EinrŸckung zeichnen
Ê       for (int i = 0; i < einrueckung; i++)
Ê       {
Ê           Console.Write(" ");
Ê       }

Ê       // Linie zeichnen
Ê       for (int i = 0; i < laenge; i++)
Ê       {
Ê           if (i == 0 || i == laenge - 1)
Ê           {
Ê               Console.Write(randZeichen + " ");
Ê           }
Ê           else
Ê           {
Ê               Console.Write(fuellZeichen + " ");
Ê           }
Ê       }
Ê       Console.WriteLine();
Ê   }

Ê   public void ZeichneRechteck(int einrueckung, int laenge, int hoehe, char fuellZeichen, char
randZeichen)
Ê   {
Ê       if (hoehe < 0)
Ê           throw new ArgumentOutOfRangeException(nameof(hoehe));

Ê       for (int zeile = 0; zeile < hoehe; zeile++)
Ê       {
Ê           if (zeile == 0 || zeile == hoehe - 1)
Ê           {
Ê               // Erste und letzte Zeile verwenden das Randzeichen Ÿberall
```

```
Ê                ZeichneLinie(einrueckung, laenge, randZeichen, randZeichen);
Ê            }
Ê            else
Ê            {
Ê                // Mittlere Zeilen verwenden das FŸllzeichen
Ê                ZeichneLinie(einrueckung, laenge, fuellZeichen, randZeichen);
Ê            }
Ê        }
Ê    }

Ê    public void ZeichneQuadrat(int einrueckung, int laenge)
Ê    {
Ê        // Ein Quadrat ist ein Spezialfall eines Rechtecks
Ê        ZeichneRechteck(einrueckung, laenge, laenge, '.', '#');
Ê    }
}
```

Kapitel 10. Arrays und Collections

10.1. Arrays

Ein Array ist ein "Feld" oder eine Anordnung mehrerer Werte des selben Datentyps, wobei die konkreten Elemente des Arrays mittels *Index* angesprochen werden.

```
// Deklaration und Initialisierung
int[] zahlen = new int[10];

// Zuweisung
zahlen[0] = 42;
zahlen[1] = 23;

// Auslesen
int ersteZahl = zahlen[0];

// LŠnge ermitteln
int anzahl = zahlen.Length;

// Iteration
for (int i = 0; i < zahlen.Length; i++)
{
Ê   Console.WriteLine(zahlen[i]);
}
```

10.2. Generische Listen (List<T>)

Im Gegensatz zu Arrays, deren Grš§e fix ist, kšnnen `List<T>` dynamisch wachsen:

```
using System.Collections.Generic;

// Erstellen
List<string> namen = new List<string>();

// HinzufŸgen
namen.Add("Max");
namen.Add("Anna");

// Zugriff (wie bei Array)
string ersterName = namen[0];

// Anzahl Elemente
int anzahl = namen.Count;

// Entfernen
namen.RemoveAt(0);  // Entfernt erstes Element
namen.Remove("Max"); // Entfernt nach Wert

// Iteration
foreach (string name in namen)
{
Ê   Console.WriteLine(name);
}
```

10.3. Weitere Collection-Typen

Typ	Beschreibung
`Dictionary<K, V>`	SchlŸssel-Wert-Paare mit schnellem Zugriff

Typ	Beschreibung
`HashSet<T>`	Unsortierte Menge ohne Duplikate
`Queue<T>`	FIFO-Warteschlange (First In, First Out) wie Supermarktkassa
`Stack<T>`	LIFO-Stapel (Last In, First Out) Kartenspiel

10.4. Manuelle Array-Verwaltung

In der professionellen Entwicklung verwenden wir meist `List<T>`. Aus didaktischen GrŸnden ist es jedoch wichtig zu verstehen, wie man die Grš§e eines Arrays manuell verwaltet. Da ein Array eine fixe Grš§e hat, mŸssen wir oft selbst mitzŠhlen, wie viele "echte" Daten bereits darin gespeichert sind.

```
public class Messreihe
{
Ê   private double[] _temperaturen;
Ê   private int _anzahl; // ZŠhler fŸr die aktuell belegten PlŠtze

Ê   public Messreihe(int maximaleKapazitaet)
Ê   {
Ê       if (maximaleKapazitaet <= 0)
Ê           throw new ArgumentException("KapazitŠt muss positiv sein");

Ê       _temperaturen = new double[maximaleKapazitaet];
Ê       _anzahl = 0;
Ê   }

Ê   public void Hinzufuegen(double temperatur)
Ê   {
Ê       // Fail-Fast: PrŸfen ob noch Platz ist
Ê       if (_anzahl >= _temperaturen.Length)
Ê           throw new InvalidOperationException("Die Messreihe ist voll!");

Ê       _temperaturen[_anzahl] = temperatur;
Ê       _anzahl++;
Ê   }

Ê   public void Einfuegen(int index, double temperatur)
Ê   {
Ê       if (index < 0 || index > _anzahl)
Ê           throw new ArgumentOutOfRangeException(nameof(index));
Ê       if (_anzahl >= _temperaturen.Length)
Ê           throw new InvalidOperationException("Kein Platz zum EinfŸgen");

Ê       // Alle Elemente nach hinten verschieben (von hinten beginnend!)
Ê       for (int i = _anzahl; i > index; i--)
Ê       {
Ê           _temperaturen[i] = _temperaturen[i - 1];
Ê       }

Ê       _temperaturen[index] = temperatur;
Ê       _anzahl++;
Ê   }

Ê   public void Entfernen(int index)
Ê   {
Ê       if (index < 0 || index >= _anzahl)
Ê           throw new ArgumentOutOfRangeException(nameof(index));
```

```
Ê       // Alle Elemente nach vorne verschieben
Ê       for (int i = index; i < _anzahl - 1; i++)
Ê       {
Ê           _temperaturen[i] = _temperaturen[i + 1];
Ê       }

Ê       _anzahl--;
Ê       _temperaturen[_anzahl] = 0; // Optional: Letzten Platz "leeren"
Ê   }

Ê   public double BerechneDurchschnitt()
Ê   {
Ê       if (_anzahl == 0) return 0;

Ê       double summe = 0;
Ê       for (int i = 0; i < _anzahl; i++)
Ê       {
Ê           summe += _temperaturen[i];
Ê       }
Ê       return summe / _anzahl;
Ê   }
}
```

Kapitel 11. Statische Methoden und NŸtzliche Klassen

11.1. Statische Methoden

Manchmal mšchte man eine Methode verwenden, ohne vorher ein Objekt der Klasse zu erzeugen. Dies ist sinnvoll, wenn die Methode keinerlei Informationen aus dem Zustand eines Objektes benštigt (keine Felder verwendet). Solche Methoden werden mit dem SchlŸsselwort `static` markiert.

```
public class Kurs
{
Ê   // Statische Methode: Gehšrt zur Klasse, nicht zum Objekt
Ê   public static bool IstZulassungMoeglich(Student student)
Ê   {
Ê       if (student == null) return false;

Ê       // Zulassung nur wenn Geburtsjahr vor 2005
Ê       return student.Geburtsjahr < 2005;
Ê   }
}

// Aufruf ohne Instanz:
bool darfTeilnehmen = Kurs.IstZulassungMoeglich(meinStudent);
```

11.2. Die `Main`-Methode

Einen wichtigen Spezialfall einer statischen Methode stellt die `Main`-Methode dar. Sie definiert den Einstiegspunkt in ein Programm. Wenn eine Anwendung gestartet wird, sucht das Betriebssystem nach dieser Methode.

```
public class Programm
{
Ê   public static void Main(string[] args)
Ê   {
Ê       Console.WriteLine("Programm gestartet!");
Ê       // Hier beginnt die AusfŸhrung
Ê   }
}
```

In modernen C#-Versionen (.NET 5+) kšnnen auch Top-Level Statements verwendet werden. Dabei schreibt man den Code direkt in die Datei `Program.cs` ohne explizite `Main`-Methode. Der Compiler erzeugt die `Main`-Methode dann automatisch im Hintergrund.

11.3. NŸtzliche Klassen: Math und Random

11.3.1. Die Klasse `Math`

Die Klasse `Math` bietet zahlreiche statische Methoden fŸr mathematische Berechnungen:

- `Math.Abs(x)`: Absolutbetrag
- `Math.Sqrt(x)`: Quadratwurzel
- `Math.Pow(x, y)`: Potenz (x^y)
- `Math.Round(x)`: Runden
- `Math.Max(a, b)` / `Math.Min(a, b)`: Maximum/Minimum

11.3.2. Die Klasse `Random`

FŸr die Erzeugung von Zufallszahlen verwenden wir die Klasse `Random`. Im Gegensatz zu Java erzeugt man in C# meist eine Instanz und verwendet diese wieder:

```
Random wuerfel = new Random();

// Zufallszahl zwischen 1 (inklusive) und 7 (exklusive) -> 1 bis 6
int ergebnis = wuerfel.Next(1, 7);

// ZufŠllige Kommazahl zwischen 0.0 und 1.0
double wahrscheinlichkeit = wuerfel.NextDouble();
```

Kapitel 12. Vererbung

Vererbung ist ein fundamentales Konzept der objektorientierten Programmierung. Eine Klasse kann von einer anderen Klasse erben und damit deren Eigenschaften und Methoden Ÿbernehmen.

```
// Basisklasse
public class Person
{
Ê   private string _name;
Ê   private int _geburtsjahr;

Ê   public Person(string name, int geburtsjahr)
Ê   {
Ê       if (string.IsNullOrWhiteSpace(name))
Ê           throw new ArgumentException("Name darf nicht leer sein", nameof(name));

Ê       _name = name;
Ê       _geburtsjahr = geburtsjahr;
Ê   }

Ê   public string Name => _name;
Ê   public int Geburtsjahr => _geburtsjahr;

Ê   public virtual int BerechneAlter(int aktuellesJahr)
Ê   {
Ê       return aktuellesJahr - _geburtsjahr;
Ê   }
}

// Abgeleitete Klasse
public class Student : Person
{
Ê   private string _matrikelnummer;

Ê   public Student(string name, int geburtsjahr, string matrikelnummer)
Ê       : base(name, geburtsjahr)
Ê   {
Ê       if (string.IsNullOrWhiteSpace(matrikelnummer))
Ê           throw new ArgumentException("Matrikelnummer darf nicht leer sein", nameof
(matrikelnummer));

Ê       _matrikelnummer = matrikelnummer;
Ê   }

Ê   public string Matrikelnummer => _matrikelnummer;

Ê   public override int BerechneAlter(int aktuellesJahr)
Ê   {
Ê       Console.WriteLine("Berechnung des Alters fŸr Student...");
Ê       return base.BerechneAlter(aktuellesJahr);
Ê   }
}
```

Das SchlŸsselwort `virtual` markiert eine Methode, die in abgeleiteten Klassen Ÿberschrieben werden kann. `override` Ÿberschreibt die Basismethode. `base` ruft die Implementierung der Basisklasse auf.

12.1. Abstrakte Klassen und Methoden

```
public abstract class Mitarbeiter
{
Ê   private string _name;

Ê   protected Mitarbeiter(string name)
Ê   {
Ê       _name = name;
Ê   }

Ê   public string Name => _name;

Ê   // Abstrakte Methode - muss in abgeleiteten Klassen implementiert werden
Ê   public abstract decimal BerechneGehalt();
}

public class Festangestellter : Mitarbeiter
{
Ê   private decimal _grundgehalt;

Ê   public Festangestellter(string name, decimal grundgehalt)
Ê       : base(name)
Ê   {
Ê       _grundgehalt = grundgehalt;
Ê   }

Ê   public override decimal BerechneGehalt()
Ê   {
Ê       return _grundgehalt;
Ê   }
}
```

Kapitel 13. Interfaces

Ein Interface definiert einen Vertrag - eine Menge von Methoden und Properties, die eine Klasse implementieren muss. Interfaces enthalten keine Implementierung.

```
public interface IRepository<T>     ist "typisiert"
{
Ê   T GetById(int id);                                                    repo
Ê   void Add(T entity);
Ê   void Update(T entity);
Ê   void Delete(int id);                                                  serivce
Ê   IEnumerable<T> GetAll();
}
                    in Java wï¿½re es "implemen
                                                                          controller/API
public class StudentRepository : IRepository<Student>
{
Ê   private readonly List<Student> _studenten = new List<Student>();

Ê   public void Add(Student student)
Ê   {
Ê       if (student == null)
Ê           throw new ArgumentNullException(nameof(student));

Ê       _studenten.Add(student);
Ê   }

Ê   public void Delete(int id)
Ê   {
Ê       var student = GetById(id);
Ê       if (student != null)
Ê           _studenten.Remove(student);
Ê   }

Ê   public Student GetById(int id)
Ê   {
Ê       return _studenten.FirstOrDefault(s => s.Id == id);
Ê   }

Ê   public void Update(Student student)
Ê   {
Ê       // Implementierung...
Ê   }

Ê   public IEnumerable<Student> GetAll()
Ê   {
Ê       return _studenten.AsReadOnly();
Ê   }
}
```

Kapitel 14. Datenbanken mit Entity Framework Core (EF Core)

Entity Framework Core (EF Core) ist ein moderner Object-Relational Mapper (ORM) fŸr .NET. Er ermšglicht es, Datenbanken mithilfe von C#-Objekten zu verwalten und Abfragen direkt in C# (via LINQ) zu schreiben.

Wir verwenden in diesem Skriptum SQLite, eine leichtgewichtige, dateibasierte Datenbank, die keine Serverinstallation erfordert.

14.1. EF Core Pakete installieren

Um EF Core mit SQLite zu nutzen, mŸssen entsprechende NuGet-Pakete installiert werden:

```
# Core Framework und SQLite Support
dotnet add package Microsoft.EntityFrameworkCore.Sqlite

# Werkzeuge fŸr Migrations (CLI Tools)
dotnet add package Microsoft.EntityFrameworkCore.Design
```

14.2. Das DomŠnenmodell und der DbContext

Das HerzstŸck von EF Core ist der `DbContext`. Er reprŠsentiert eine Sitzung mit der Datenbank und ermšglicht das Abfragen und Speichern von Daten.

```
using Microsoft.EntityFrameworkCore;

// DomŠnenmodell
public class Student
{
Ê   public int Id { get; set; } // EF Core erkennt "Id" automatisch als PrimŠrschlŸssel
Ê   public string Name { get; set; } = string.Empty;
Ê   public int Geburtsjahr { get; set; }
}

// Datenbankkontext
public class SchulContext : DbContext
{
Ê   public DbSet<Student> Studenten { get; set; } = null!;

Ê   protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
Ê   {
Ê       // Verbindung zur SQLite-Datei herstellen
Ê       optionsBuilder.UseSqlite("Data Source=schule.db");
Ê   }
}
```

14.3. Schema-Migrations

Migrations ermšglichen es, das Datenbank-Schema synchron mit den C#-Klassen zu halten. Jede €nderung am Modell (z.B. ein neues Feld) wird in eine Migrations-Datei Ÿbersetzt.

14.3.1. Die erste Migration erstellen

Nachdem der `DbContext` definiert wurde, muss die Datenbank initialisiert werden:

```
# Erstellt den Code fŸr das Tabellenschema
dotnet ef migrations add InitialCreate
```

```
# Wendet die Migration auf die Datenbank an (erzeugt schule.db)
dotnet ef database update
```

Hinweis: Sollte der Befehl `dotnet ef` nicht gefunden werden, muss das globale Tool installiert werden: `dotnet tool install --global dotnet-ef`.

14.3.2. Schema-€nderungen durchfŸhren

Wenn Sie eine Klasse Šndern (z.B. ein Feld `Email` zu `Student` hinzufŸgen), wiederholen Sie den Vorgahng: 1. `dotnet ef migrations add AddStudentEmail` 2. `dotnet ef database update`

14.4. CRUD-Operationen

Dank EF Core kšnnen Datenbankoperationen wie normale Listenoperationen durchgefŸhrt werden:

```
using var context = new SchulContext();

// 1. Create (HinzufŸgen)
var neuerStudent = new Student { Name = "Max Mustermann", Geburtsjahr = 2005 };
context.Studenten.Add(neuerStudent);
context.SaveChanges(); // WICHTIG: Erst hier wird der SQL-Befehl ausgefŸhrt!

// 2. Read (Abfragen)
var student = context.Studenten
Ê   .FirstOrDefault(s => s.Name == "Max Mustermann");

// 3. Update (Aktualisieren)
if (student != null)
{
Ê   student.Geburtsjahr = 2006;
Ê   context.SaveChanges();
}

// 4. Delete (Lšschen)
if (student != null)
{
Ê   context.Studenten.Remove(student);
Ê   context.SaveChanges();
}
```

14.5. 1:n Beziehungen (One-to-Many)

In einer 1:n Beziehung kann ein Datensatz der Tabelle A mit mehreren DatensŠtzen der Tabelle B verknŸpft sein, aber ein Datensatz der Tabelle B gehšrt zu genau einem Datensatz der Tabelle A. Ein klassisches Beispiel ist die Beziehung zwischen einer Abteilung und ihren Mitarbeitern.

```
public class Abteilung
{
Ê   public int Id { get; set; }
Ê   public string Name { get; set; } = string.Empty;

Ê   // Navigation Property: Eine Abteilung hat viele Mitarbeiter
Ê   public List<Mitarbeiter> Mitarbeiter { get; set; } = new();
}

public class Mitarbeiter
```

```
{
Ê   public int Id { get; set; }
Ê   public string Name { get; set; } = string.Empty;

Ê   // FremdschlŸssel (Foreign Key)
Ê   public int AbteilungId { get; set; }

Ê   // Navigation Property: Ein Mitarbeiter gehšrt zu einer Abteilung
Ê   public Abteilung Abteilung { get; set; } = null!;
}
```

EF Core erkennt diese Beziehung automatisch anhand der Navigation Properties und des FremdschlŸssels (`AbteilungId`).

14.6. n:m Beziehungen (Many-to-Many)

In einer n:m Beziehung kšnnen mehrere DatensŠtze der Tabelle A mit mehreren DatensŠtzen der Tabelle B verknŸpft sein. Ein Beispiel hierfŸr ist die Beziehung zwischen Studenten und Kursen: Ein Student kann viele Kurse besuchen, und ein Kurs kann von vielen Studenten besucht werden.

```
public class Student
{
Ê   public int Id { get; set; }
Ê   public string Name { get; set; } = string.Empty;

Ê   // Ein Student besucht viele Kurse
Ê   public List<Kurs> Kurse { get; set; } = new();
}

public class Kurs
{
Ê   public int Id { get; set; }
Ê   public string Titel { get; set; } = string.Empty;

Ê   // Ein Kurs hat viele Studenten
Ê   public List<Student> Studenten { get; set; } = new();
}
```

Seit EF Core 5.0 erstellt das Framework automatisch eine Zwischentabelle (Join-Table) in der Datenbank, ohne dass diese explizit als Klasse im C#-Code definiert werden muss.

14.7. Laden von Beziehungen (Eager Loading)

StandardmŠ§ig lŠdt EF Core keine verwandten Daten (Lazy Loading ist standardmŠ§ig deaktiviert). Um Beziehungen mitzuladen, verwenden wir die `Include`-Methode. Dies wird als Eager Loading bezeichnet.

```
using Microsoft.EntityFrameworkCore;

// Mitarbeiter inklusive ihrer Abteilung laden
var mitarbeiterListe = context.Mitarbeiter
Ê   .Include(m => m.Abteilung)
Ê   .ToList();

// Abfrage mit Filterung Ÿber die Beziehung
var itMitarbeiter = context.Mitarbeiter
Ê   .Include(m => m.Abteilung)
```

```
Ê    .Where(m => m.Abteilung.Name == "IT")
Ê    .ToList();
```

14.8. Best Practices fŸr EF Core

1. Unit of Work: Verwenden Sie `SaveChanges()` gesammelt am Ende einer logischen Operation, nicht nach jeder einzelnen Zeile.
2. AsNoTracking: Verwenden Sie `.AsNoTracking()`, wenn Sie Daten nur lesen mšchten (erhšht die Performance).
3. Fail-Fast: Validieren Sie Daten bereits im DomŠnenmodell, bevor Sie `SaveChanges()` aufrufen.
4. Migrationen einchecken: Die Migrations-Dateien im Verzeichnis `Migrations/` gehšren in das Git-Repository, da sie die Historie des Schemas dokumentieren.

Kapitel 15. LINQ (Language Integrated Query)

LINQ ist ein mŠchtiges Feature von C#, das es ermšglicht, Daten aus verschiedenen Quellen (Collections, Datenbanken, XML, etc.) auf einheitliche Weise zu abzufragen und zu manipulieren.

15.1. Grundlegende LINQ-Operationen

```
using System.Linq;

List<int> zahlen = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

// Filterung (Where)
var geradeZahlen = zahlen.Where(z => z % 2 == 0);

// Projektion (Select)
var quadrate = zahlen.Select(z => z * z);

// Sortierung (OrderBy, OrderByDescending)
var absteigend = zahlen.OrderByDescending(z => z);

// Aggregationen
int summe = zahlen.Sum();
double durchschnitt = zahlen.Average();
int maximum = zahlen.Max();
int minimum = zahlen.Min();
int anzahl = zahlen.Count();

// ExistenzprŸfungen
bool hatGerade = zahlen.Any(z => z % 2 == 0);
bool allePositiv = zahlen.All(z => z > 0);

// Erstes Element
int ersteGerade = zahlen.First(z => z % 2 == 0);
int ersteOderDefault = zahlen.FirstOrDefault(z => z > 100); // 0 wenn nicht gefunden

// Selektion (Findet ein einzelnes Element)
int dieFuenf = zahlen.Single(z => z == 5);
```

15.2. LINQ with Objects

```
public class Student
{
Ê   public int Id { get; set; }
Ê   public string Name { get; set; }
Ê   public int Geburtsjahr { get; set; }
Ê   public string Studiengang { get; set; }
}

// Beispielabfragen
List<Student> studenten = LadeStudenten();

// Alle Studenten eines Studiengangs
var informatikStudenten = studenten
Ê   .Where(s => s.Studiengang == "Informatik")
Ê   .OrderBy(s => s.Name);

// Namen aller Studenten
var namen = studenten.Select(s => s.Name);

// Gruppierung nach Studiengang
```

```
var gruppiert = studenten
Ê   .GroupBy(s => s.Studiengang)
Ê   .Select(g => new
Ê   {
Ê       Studiengang = g.Key,
Ê       Anzahl = g.Count(),
Ê       DurchschnittsAlter = DateTime.Now.Year - g.Average(s => s.Geburtsjahr)
Ê   });

// Komplexe Abfrage mit mehreren Operationen
var ergebnis = studenten
Ê   .Where(s => s.Geburtsjahr > 1990)
Ê   .OrderBy(s => s.Name)
Ê   .Take(10)
Ê   .Select(s => $"{s.Name} ({DateTime.Now.Year - s.Geburtsjahr} Jahre)")
Ê   .ToList();
```

15.3. Query-Syntax vs. Method-Syntax

LINQ bietet zwei Syntax-Varianten:

```
// Method-Syntax (Lambda-Syntax)
var ergebnis1 = studenten
Ê   .Where(s => s.Geburtsjahr > 1990)
Ê   .OrderBy(s => s.Name)
Ê   .Select(s => s.Name);

// Query-Syntax (SQL-Šhnlich)
var ergebnis2 = from s in studenten
Ê               where s.Geburtsjahr > 1990
Ê               orderby s.Name
Ê               select s.Name;

// Beide Varianten sind Šquivalent!
```

Kapitel 16. Exception Handling

16.1. Grundlagen

Exceptions (Ausnahmen) sind Fehler, die zur Laufzeit auftreten. In C# werden Exceptions mit `try-catch-finally` behandelt:

```
public void BeispielMethode()
{
Ê   try
Ê   {
Ê       // Code, der eine Exception werfen kšnnte
Ê       int ergebnis = 10 / 0;
Ê   }
Ê   catch (DivideByZeroException ex)
Ê   {
Ê       // Spezifische Behandlung fŸr Division durch Null
Ê       Console.WriteLine($"Fehler: {ex.Message}");
Ê   }
Ê   catch (Exception ex)
Ê   {
Ê       // Allgemeine Behandlung
Ê       Console.WriteLine($"Unbekannter Fehler: {ex.Message}");
Ê   }
Ê   finally
Ê   {
Ê       // Wird immer ausgefŸhrt, auch wenn eine Exception auftrat
Ê       // Gut fŸr Cleanup-Operationen
Ê   }
}
```

16.2. Eigene Exceptions

```
public class ValidationException : Exception
{
Ê   public ValidationException(string message) : base(message)
Ê   {
Ê   }

Ê   public ValidationException(string message, Exception innerException)
Ê       : base(message, innerException)
Ê   {
Ê   }
}

public class StudentService
{
Ê   public void RegistriereStudent(Student student)
Ê   {
Ê       if (student == null)
Ê           throw new ArgumentNullException(nameof(student));

Ê       if (string.IsNullOrWhiteSpace(student.Name))
Ê           throw new ValidationException("Studentenname ist erforderlich");

Ê       // Registrierung...
Ê   }
}
```

Kapitel 17. Razor

Razor ist eine Syntax fŸr die Erstellung dynamischer Webseiten mit C#. Sie ermšglicht die nahtlose Einbettung von C#-Code in HTML.

17.1. Razor-Grundlagen

Eine Razor-Datei hat die Endung `.cshtml`. Sie enthŠlt HTML mit eingebettetem C#-Code:

```
@page
@model IndexModel

<!DOCTYPE html>
<html>
<head>
Ê   <title>@Model.Seitentitel</title>
</head>
<body>
Ê   <h1>Willkommen, @Model.Benutzername!</h1>

Ê   @if (Model.IstAngemeldet)
Ê   {
Ê       <p>Sie sind angemeldet.</p>
Ê   }
Ê   else
Ê   {
Ê       <p>Bitte melden Sie sich an.</p>
Ê   }

Ê   <ul>
Ê   @foreach (var eintrag in Model.Eintraege)
Ê   {
Ê       <li>@eintrag.Name - @eintrag.Datum.ToShortDateString()</li>
Ê   }
Ê   </ul>
</body>
</html>
```

17.2. Razor-Page Model

Das Page Model enthŠlt die Logik fŸr eine Razor-Page:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

public class IndexModel : PageModel
{
Ê   private readonly IStudentService _studentService;

Ê   public IndexModel(IStudentService studentService)
Ê   {
Ê       _studentService = studentService ?? throw new ArgumentNullException(nameof
(studentService));
Ê   }

Ê   public string Seitentitel { get; set; } = "Meine Seite";
Ê   public string Benutzername { get; set; }
Ê   public bool IstAngemeldet { get; set; }
Ê   public List<Eintrag> Eintraege { get; set; } = new List<Eintrag>();

Ê   public void OnGet()
```

```
Ê   {
Ê       // Wird bei GET-Request aufgerufen
Ê       Eintraege = _studentService.LadeAlleEintraege();
Ê   }

Ê   public IActionResult OnPost(int id)
Ê   {
Ê       // Wird bei POST-Request aufgerufen
Ê       try
Ê       {
Ê           _studentService.LoescheEintrag(id);
Ê           return RedirectToPage("/Index");
Ê       }
Ê       catch (Exception ex)
Ê       {
Ê           ModelState.AddModelError("", $"Fehler beim Lšschen: {ex.Message}");
Ê           return Page();
Ê       }
Ê   }
}
```

17.3. Razor-Komponenten

Razor-Komponenten (Blazor) ermšglichen die Erstellung wiederverwendbarer UI-Elemente:

```
<!-- StudentCard.razor -->
<div class="student-card">
Ê   <h3>@Student.Name</h3>
Ê   <p>Matrikelnummer: @Student.Matrikelnummer</p>
Ê   <button @onclick="OnDetailsClick">Details</button>
</div>

@code {
Ê   [Parameter]
Ê   public Student Student { get; set; }

Ê   [Parameter]
Ê   public EventCallback<int> OnDetails { get; set; }

Ê   private async Task OnDetailsClick()
Ê   {
Ê       await OnDetails.InvokeAsync(Student.Id);
Ê   }
}
```

Kapitel 18. Razor mit HTMX

HTMX ist eine JavaScript-Bibliothek, die es ermšglicht, moderne interaktive Webanwendungen zu erstellen, ohne komplexes JavaScript zu schreiben. In Kombination mit Razor erlaubt es die Erstellung von Server-seitig gerenderten Webanwendungen mit dynamischem Verhalten.

18.1. Was ist HTMX?

HTMX erweitert HTML um Attribute, die AJAX-Requests, CSS-Transitions und WebSocket-Verbindungen direkt im Markup ermšglichen:

```
<!-- Standard HTML-Button -->
<button>Klick mich</button>

<!-- HTMX-erweiterter Button -->
<button hx-get="/api/studenten"
Ê       hx-target="#studenten-liste"
Ê       hx-trigger="click">
Ê   Studenten laden
</button>
```

18.2. HTMX in Razor Pages

```
@page
@model StudentenModel

<h1>Studentenverwaltung</h1>

<!-- Teil der Seite, der dynamisch aktualisiert wird -->
<div id="studenten-container">
Ê   <partial name="_StudentenListe" model="Model.Studenten" />
</div>

<!-- Formular mit HTMX fŸr serverseitige Verarbeitung -->
<form hx-post="/Studenten"
Ê     hx-target="#studenten-container"
Ê     hx-swap="innerHTML">

Ê   <div>
Ê       <label for="name">Name:</label>
Ê       <input type="text" id="name" name="Name" required />
Ê   </div>

Ê   <div>
Ê       <label for="geburtsjahr">Geburtsjahr:</label>
Ê       <input type="number" id="geburtsjahr" name="Geburtsjahr" required />
Ê   </div>

Ê   <button type="submit">Student hinzufŸgen</button>
</form>

<!-- Lšschen mit BestŠtigung -->
@foreach (var student in Model.Studenten)
{
Ê   <div class="student-eintrag">
Ê       <span>@student.Name</span>
Ê       <button hx-delete="/Studenten?id=@student.Id"
Ê               hx-target="#studenten-container"
Ê               hx-confirm="Mšchten Sie @student.Name wirklich lšschen?"
Ê               hx-swap="outerHTML">
```

```
Ê            Lšschen
Ê        </button>
Ê    </div>
}
```

18.3. PageModel fŸr HTMX

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

public class StudentenModel : PageModel
{
Ê    private readonly IStudentRepository _repository;

Ê    public StudentenModel(IStudentRepository repository)
Ê    {
Ê        _repository = repository ?? throw new ArgumentNullException(nameof(repository));
Ê    }

Ê    public List<Student> Studenten { get; set; } = new List<Student>();

Ê    public void OnGet()
Ê    {
Ê        Studenten = _repository.GetAll();
Ê    }

Ê    public IActionResult OnPost(StudentInput input)
Ê    {
Ê        if (!ModelState.IsValid)
Ê        {
Ê            return Partial("_StudentenListe", _repository.GetAll());
Ê        }

Ê        try
Ê        {
Ê            var student = new Student(input.Name, input.Geburtsjahr);
Ê            _repository.Add(student);

Ê            // Nur den aktualisierten Teil zurŸckgeben
Ê            return Partial("_StudentenListe", _repository.GetAll());
Ê        }
Ê        catch (Exception ex)
Ê        {
Ê            ModelState.AddModelError("", ex.Message);
Ê            return Partial("_StudentenListe", _repository.GetAll());
Ê        }
Ê    }

Ê    public IActionResult OnDelete(int id)
Ê    {
Ê        try
Ê        {
Ê            _repository.Delete(id);
Ê            return Partial("_StudentenListe", _repository.GetAll());
Ê        }
Ê        catch (Exception ex)
Ê        {
Ê            // HTMX wird den Fehler anzeigen
Ê            return StatusCode(500, ex.Message);
Ê        }
Ê    }
```

```
}

public class StudentInput
{
Ê   public string Name { get; set; }
Ê   public int Geburtsjahr { get; set; }
}
```

18.4. HTMX-Partial Views

```
<!-- _StudentenListe.cshtml -->
@model List<Student>

<div id="studenten-container">
Ê   @if (Model.Any())
Ê   {
Ê       <table class="table">
Ê           <thead>
Ê               <tr>
Ê                   <th>Name</th>
Ê                   <th>Geburtsjahr</th>
Ê                   <th>Aktionen</th>
Ê               </tr>
Ê           </thead>
Ê           <tbody>
Ê               @foreach (var student in Model)
Ê               {
Ê                   <tr>
Ê                       <td>@student.Name</td>
Ê                       <td>@student.Geburtsjahr</td>
Ê                       <td>
Ê                           <button hx-delete="/Studenten?id=@student.Id"
Ê                                   hx-target="#studenten-container"
Ê                                   hx-confirm="Wirklich lšschen?"
Ê                                   class="btn btn-danger">
Ê                               Lšschen
Ê                           </button>
Ê                       </td>
Ê                   </tr>
Ê               }
Ê           </tbody>
Ê       </table>
Ê   }
Ê   else
Ê   {
Ê       <p>Keine Studenten vorhanden.</p>
Ê   }
</div>
```

18.5. HTMX-Trigger und Events

HTMX bietet vielfŠltige Trigger-Optionen:

```
<!-- Trigger beim Laden der Seite -->
<div hx-get="/api/status"
Ê    hx-trigger="load"
Ê    hx-target="this">
Ê   Lade Status...
</div>

<!-- Trigger bei Eingabe (debounced) -->
```

```
<input type="search"
Ê      name="suchbegriff"
Ê      hx-get="/api/suche"
Ê      hx-trigger="keyup changed delay:500ms"
Ê      hx-target="#suchergebnisse"
Ê      placeholder="Suchen..." />

<!-- Trigger in Intervallen -->
<div hx-get="/api/aktuelle-uhrzeit"
Ê    hx-trigger="every 5s"
Ê    hx-target="this">
Ê   @DateTime.Now.ToString("HH:mm:ss")
</div>

<!-- Trigger bei Sichtbarwerden -->
<div hx-get="/api/mehr-daten"
Ê    hx-trigger="revealed"
Ê    hx-target="this">
Ê   Mehr laden...
</div>
```

18.6. HTMX mit Animationen

```
<!-- CSS fŸr Transition -->
<style>
.student-eintrag {
Ê   transition: all 0.3s ease;
Ê   opacity: 1;
}

.student-eintrag.htmx-swapping {
Ê   opacity: 0;
Ê   transform: translateX(20px);
}

.student-eintrag.htmx-added {
Ê   opacity: 0;
Ê   transform: translateX(-20px);
}
</style>

<!-- HTML mit HTMX -->
<div id="studenten-liste" hx-swap="innerHTML swap:0.3s">
Ê   <!-- Inhalt wird hier dynamisch geladen -->
</div>

<!-- Formular mit Feedback -->
<form hx-post="/Studenten"
Ê     hx-target="#studenten-liste"
Ê     hx-swap="beforeend"
Ê     hx-on::after-request="this.reset()">

Ê   <input type="text" name="Name" required />
Ê   <button type="submit">HinzufŸgen</button>
</form>
```

18.7. Vorteile von Razor + HTMX

1. Server-seitiges Rendering: Keine komplexe Client-Logik nštig
2. Progressive Enhancement: Funktioniert auch ohne JavaScript

3. Einfachheit: Weniger Code als bei SPAs (Single Page Applications)
4. Performance: Nur Teile der Seite werden neu geladen
5. C#-Fokus: Volle Nutzung der C#-...kosystem-StŠrken

18.8. Best Practices fŸr Razor + HTMX

1. Partial Views verwenden: Erstellen Sie wiederverwendbare Partial Views fŸr dynamische Teile
2. Fail-Fast beibehalten: Validieren Sie auch in HTMX-Requests sofort
3. RŸckgabecodes: Verwenden Sie passende HTTP-Statuscodes (200, 400, 500)
4. UX beachten: Zeigen Sie LadezustŠnde und Fehlermeldungen an
5. Testbarkeit: Testen Sie die Endpunkte wie normale API-Controller

```
// Beispiel fŸr gute HTMX-Integration mit Fail-Fast
public class HtmxStudentController : Controller
{
Ê   private readonly IStudentRepository _repository;

Ê   public HtmxStudentController(IStudentRepository repository)
Ê   {
Ê       _repository = repository ?? throw new ArgumentNullException(nameof(repository));
Ê   }

Ê   [HttpPost]
Ê   [ValidateAntiForgeryToken]
Ê   public IActionResult Erstelle(StudentInput input)
Ê   {
Ê       // Fail-Fast: Validierung
Ê       if (!ModelState.IsValid)
Ê           return BadRequest(ModelState);

Ê       try
Ê       {
Ê           var student = new Student(input.Name, input.Geburtsjahr);
Ê           _repository.Add(student);

Ê           // Nur die neue Zeile zurŸckgeben
Ê           return PartialView("_StudentRow", student);
Ê       }
Ê       catch (ArgumentException ex)
Ê       {
Ê           return BadRequest(ex.Message);
Ê       }
Ê       catch (Exception ex)
Ê       {
Ê           // Logging...
Ê           return StatusCode(500, "Interner Serverfehler");
Ê       }
Ê   }
}
```